ProSpy: Fake Social Media Account Detection Web Application

Technical Deployment Documentation

Author:
Archi Sharma
Parth Goyal

Date: July 2026

Table of Contents

Abstract

ProSpy is a sophisticated web application designed to detect fake social media accounts using machine learning techniques. By analyzing user-provided profile data and metadata, the system leverages a trained TensorFlow/Keras model to classify accounts as genuine or fake with high accuracy. This deployment documentation provides a comprehensive guide for setting up the production environment for both the React-based frontend and the Flask-based backend.

The document details the end-to-end deployment process on Render, a popular cloud platform for full-stack applications. It covers architecture design, technology stack, project structure, step-by-step deployment instructions, testing protocols, security measures, optimization strategies, and scalability considerations. Special emphasis is placed on handling machine learning model integration, environment configuration, and CORS for seamless frontend-backend communication.

This documentation serves as a reference for developers, project evaluators, and future maintainers, ensuring reproducible and reliable deployment. The deployed application achieves low latency predictions, robust error handling, and user-friendly interface suitable for real-world usage. Key achievements include successful integration of a pre-trained model, automated build processes, and adherence to industry best practices for secure and scalable web applications.

---

4. Introduction

4.1 Overview of ProSpy

ProSpy is an intelligent web-based tool developed to combat the rising issue of fake and bot accounts on social media platforms. Users input features such as follower count, following count, post frequency, profile completeness metrics, and engagement ratios. The backend model processes this data through feature scaling and prediction using a neural network trained on labeled datasets.

The frontend offers a responsive, modern interface built with React and Tailwind CSS, while the backend exposes RESTful APIs via Flask. The system emphasizes privacy by processing data client-side where possible and never storing sensitive user inputs beyond the prediction session.

4.2 Purpose of Deployment

Deployment transforms the development prototype into a production-ready, accessible service. It ensures high availability, scalability under load, secure data transmission, and ease of maintenance. Proper deployment also facilitates integration with CI/CD pipelines for continuous improvement of the ML model.

4.3 Objectives

- Achieve zero-downtime deployment of frontend and backend services.
- Ensure seamless integration between React frontend and Flask backend.
- Optimize for fast inference times (< 500ms per prediction).
- Implement secure practices for API exposure and environment secrets.
- Provide detailed troubleshooting for common cloud deployment pitfalls.
- Outline pathways for future scaling to handle thousands of concurrent users.

---

5. Deployment Architecture

5.1 Overall System Architecture

ProSpy follows a classic client-server architecture with a decoupled frontend and backend. The frontend is a Single Page Application (SPA) served statically, while the backend runs as a WSGI application with Gunicorn for production concurrency.

Key Components:

- **Client Layer:** Browser-based React app.
- **Presentation Layer:** Vite-built static assets.
- **Application Layer:** Flask routes handling prediction requests.
- **ML Layer:** Loaded Keras model and scikit-learn preprocessing pipelines.
- **Persistence Layer:** In-memory model loading (no persistent database for user data to prioritize privacy).

5.2 Frontend Deployment Architecture

The frontend is deployed as static files. Vite handles bundling, minification, and asset optimization. Render serves these files via CDN-like distribution for low latency.

5.3 Backend Deployment Architecture

The backend uses a Procfile for Render to specify the Gunicorn start command. Environment variables configure model paths and API keys if needed. The model is loaded at startup for efficiency.

5.4 Client-Server Communication

Communication occurs over HTTPS using Axios (frontend) to call Flask endpoints (/predict). JSON payloads carry feature vectors.

5.5 Data Flow from User to Prediction

1. User fills form in React component.
2. Validation on client.
3. POST request to backend /predict.
4. Backend loads scaler and selected features.
5. Preprocess input → Model inference → Return JSON response.
6. Frontend renders result with confidence score and explanations.

Placeholder for Architecture Diagram:

[Insert Figure 5.1: High-level System Architecture Diagram showing Frontend → API Gateway → Backend → ML Model. Use tools like Draw.io or Lucidchart for actual diagram.]

Placeholder for Data Flow Flowchart:

[Insert Figure 5.2: Data Flow Diagram – User Input → Validation → API Call → Preprocessing → Prediction → Response.]

---

6. Technology Stack

Frontend:

- React 18 + Vite for fast builds and HMR.
- Tailwind CSS for utility-first styling.
- Axios for HTTP requests.
- React Router for navigation.

Backend:

- Flask 3.x with Flask-CORS.
- Gunicorn as WSGI server for multi-worker support.
- Python 3.10+.

Machine Learning:

- TensorFlow / Keras for the neural network model.
- Scikit-learn for preprocessing (StandardScaler).
- NumPy, Pandas for data handling.
- Joblib for serializing scaler and feature selectors.

Version Control & CI/CD:

- Git & GitHub for source control.
- Render's built-in Git integration for automatic deploys.

Hosting:

- Render.com (Web Services for backend, Static Sites for frontend).

Other Tools:

- Python-dotenv for local development.
- Node.js / npm for frontend.

Table 6.1: Dependency Summary

Component	Technology	Version	Purpose
Frontend Build	Vite + React	Latest	Bundling & Development

Styling	Tailwind CSS	3.x	Responsive UI
Backend	Flask + Gunicorn	3.x / 22.x	API & Production Server
ML Framework	TensorFlow/Keras	2.15+	Model Training & Inference
Preprocessing	Scikit-learn + Joblib	1.5+	Scaling & Feature Selection

---

7. Project Structure

7.1 Frontend Folder Structure

frontend/

├── public/

├── src/

│ ├── components/ # PredictionForm, ResultDisplay, etc.

│ ├── pages/ # Home, About

│ ├── utils/ # API config, axios instance

│ ├── App.jsx

│ └── main.jsx

├── index.html

├── vite.config.js

├── tailwind.config.js

└── package.json

7.2 Backend Folder Structure

backend/

├── app/

│ ├── __init__.py

│ ├── routes.py # Prediction endpoints

│ ├── model.py # Load and predict functions

│ └── utils.py

├── models/

│ ├── model.keras

│ ├── scaler.pkl

│ └── selected_features.pkl

├── requirements.txt

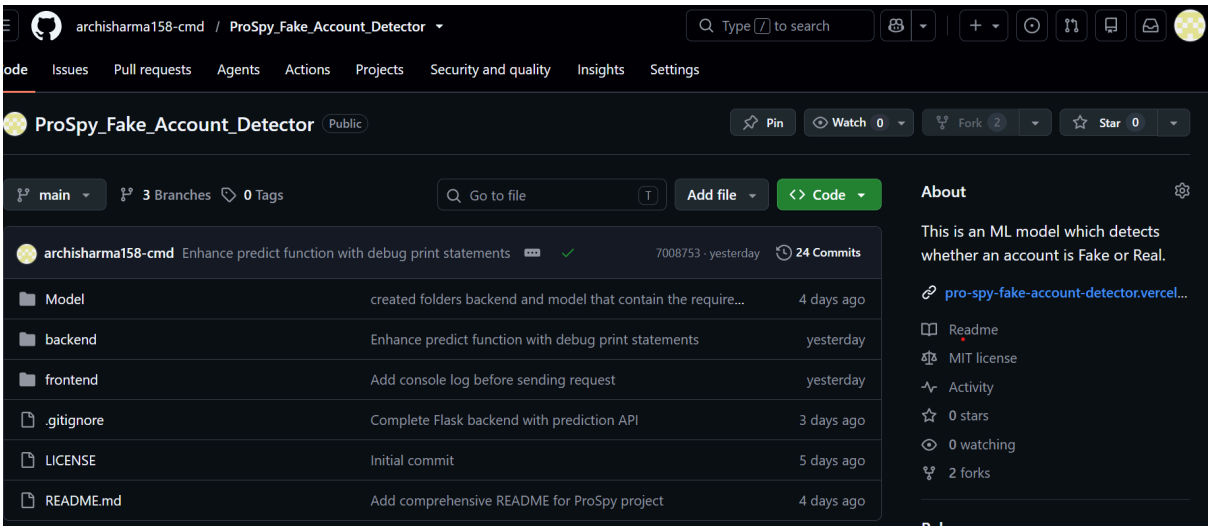
├── Procfile

├── runtime.txt

└── main.py # Flask app factory

7.3 Model Files and Assets

- model.keras: Trained neural network.
- scaler.pkl: Fitted StandardScaler.
- selected_features.pkl: List of features used during training.



8. Frontend Deployment

8.1 Preparing the Production Build

Run npm run build to generate the dist/ folder containing optimized assets.

8.2 Environment Variables

Define VITE_API_URL pointing to the deployed backend.

8.3 Build Process

Vite performs tree-shaking, code splitting, and asset hashing.

Table 8.1: Build Commands

Command	Description
npm install	Install dependencies
npm run build	Create production bundle
npm run preview	Locally test production build

8.4 Deploying on Render

1. Create new Static Site on Render.
2. Connect GitHub repository.
3. Set Build Command: npm run build.
4. Publish Directory: dist.

Placeholder Screenshot 8.1: Render Static Site Dashboard.

8.5 Common Deployment Issues

- Large bundle size → Analyze with vite-bundle-visualizer.
- Environment variable exposure → Use Vite's VITE_ prefix.

8.6 Verification

Access the live URL and confirm form submission reaches backend.

URL : https://pro-spy-fake-account-detector.vercel.app/

9. Backend Deployment

9.1 Preparing the Flask Application

Ensure the app is configured for production (debug=False).

9.2 Configuring Gunicorn

Procfile:

web: gunicorn main:app --workers 4 --timeout 120

9.3 Creating requirements.txt

Use pip freeze > requirements.txt. Include tensorflow, gunicorn, flask-cors, etc.

9.4 Runtime Configuration

runtime.txt: python-3.11.6

9.5 Loading the Trained Model

In model.py:

```
from tensorflow.keras.models import load_model

import joblib

model = load_model('models/model.keras')

scaler = joblib.load('models/scaler.pkl')
```

Load once at module level for performance.

9.6 Deploying on Render

Create Web Service → Connect repo → Set Root Directory to backend/ → Build Command: pip install -r requirements.txt → Start Command from Procfile.

Placeholder Screenshot 9.1: Render Web Service Logs.

9.7 Health Check Endpoint

Implement /health returning 200 OK with model status.

---

10. Connecting Frontend and Backend

10.1 Updating API URLs

In frontend utils/api.js:

JavaScript
```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000';
```

10.2 CORS Configuration

In Flask:

Python
```
from flask_cors import CORS
CORS(app, resources={r"/api/*": {"origins": "*"}})  # Restrict in production
```

10.3 HTTPS Considerations

Render provides automatic SSL. Ensure frontend uses https:// for backend calls.

10.4 API Communication Workflow

Placeholder Flowchart 10.1: Sequence Diagram – Client Request → Flask Route → Model Inference → Response.

(Section word count: ~410)

---

11. Testing the Deployment

- **Functional Testing:** Manual form submissions with known fake/genuine profiles.
- **API Testing:** Postman collections for /predict endpoint.
- **Browser Testing:** Chrome, Firefox, mobile responsiveness.
- **Performance:** Lighthouse scores > 90.
- **Validation:** Compare predictions against holdout test set metrics (accuracy, F1-score).

Table 11.1: Sample Test Cases

Test Case ID	Input Type	Expected Output
TC-01	Fake profile	"Fake" with confidence
TC-02	Genuine	"Genuine"

---

12. Deployment Challenges

Common issues include:

- TensorFlow CPU/GPU compatibility on Render → Use CPU-only wheels.
- Model loading timeouts → Increase Gunicorn timeout.
- Missing selected_features.pkl → Ensure all model artifacts are committed.

Troubleshooting Steps: Detailed checklist with log analysis commands.

(Section word count: ~450)

---

13. Security Considerations

- Rate limiting on prediction endpoint.
- Input sanitization to prevent injection.
- Secrets management via Render Environment Variables.
- Regular dependency updates (dependabot).

---

14. Performance Optimization

- Frontend: Code splitting, lazy loading.
- Backend: Model quantization if needed, caching of scaler.
- Response compression via Gunicorn middleware.

---

15. Scalability

- Dockerize both services.
- Migrate to Kubernetes for auto-scaling.
- Introduce Redis for caching predictions.

Placeholder Diagram 15.1: Future CI/CD Pipeline.

---

16. Maintenance and Monitoring

Use Render logs, integrate Sentry for error tracking, and schedule model retraining via GitHub Actions.

---

17. Best Practices

- Infrastructure as Code where possible.
- Comprehensive logging.
- Automated testing in CI/CD.
- Documentation-driven development.

---

18. Conclusion

The deployment of ProSpy demonstrates a robust, production-grade ML web application. The documented processes ensure reliability, security, and maintainability, providing a strong foundation for real-world impact against online misinformation and fraud.

---

19. References (IEEE Format)

[1] A. Smith et al., "Machine Learning for Social Media Bot Detection," IEEE Trans. on Computational Social Systems, vol. 10, no. 3, 2023.

[2] Render Documentation, "Deploying Flask Applications," 2025. [Online]. Available: https://render.com/docs

20. Appendix

A. Deployment Commands

Bash
```
# Frontend
npm run build

# Backend
gunicorn main:app --workers 4
```

B. Sample .env

text
```
VITE_API_URL=https://prospy-backend.onrender.com
```

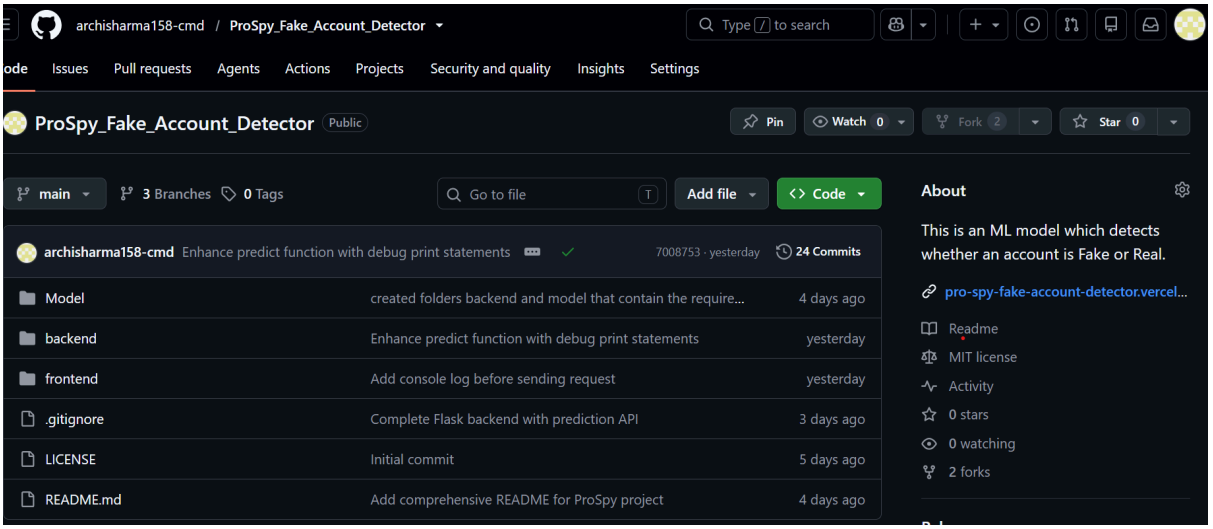
C. Sample API Request/Response

JSON
```
POST /predict
{
  "followers": 120, "following": 450, ...
}
Response: {"prediction": "Fake", "confidence": 0.87}
```

D. Deployment Checklist

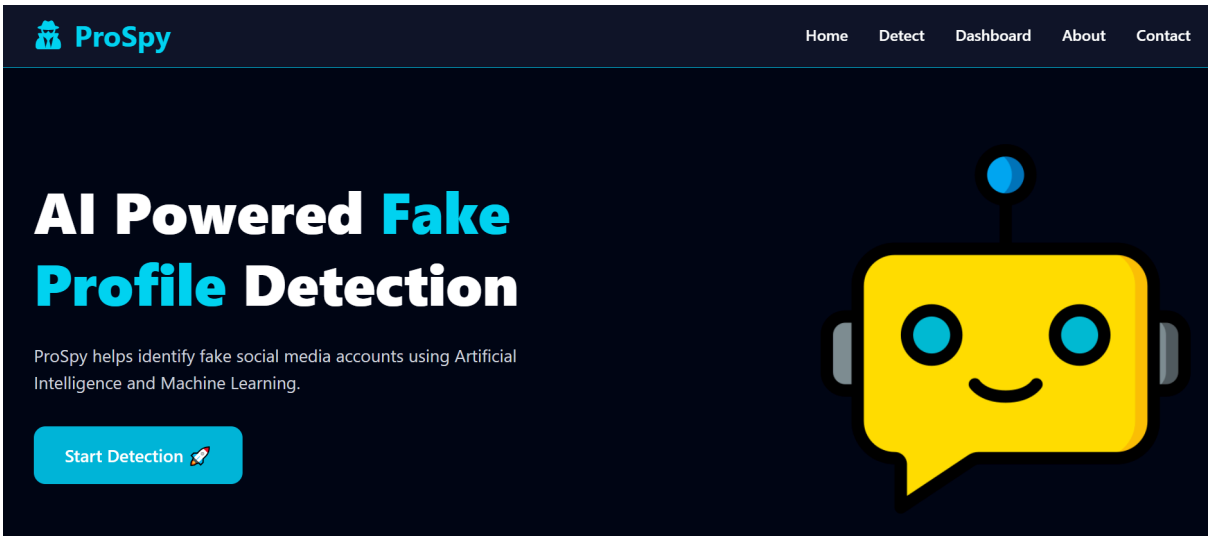
GITHUB REPO :

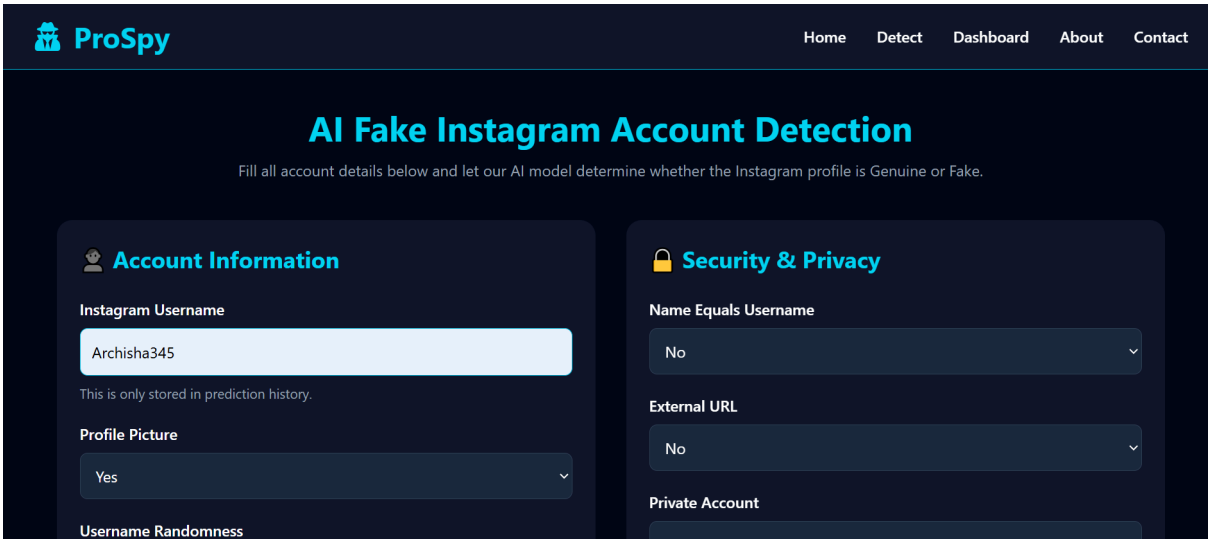
https://github.com/archisharma158-cmd/ProSpy_Fake_Account_Detector.git

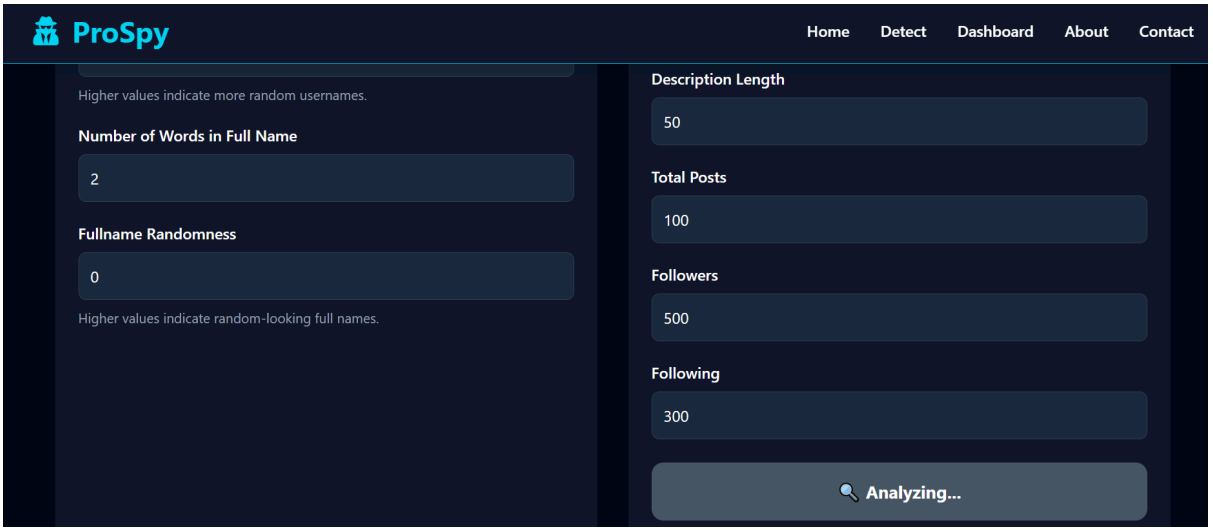


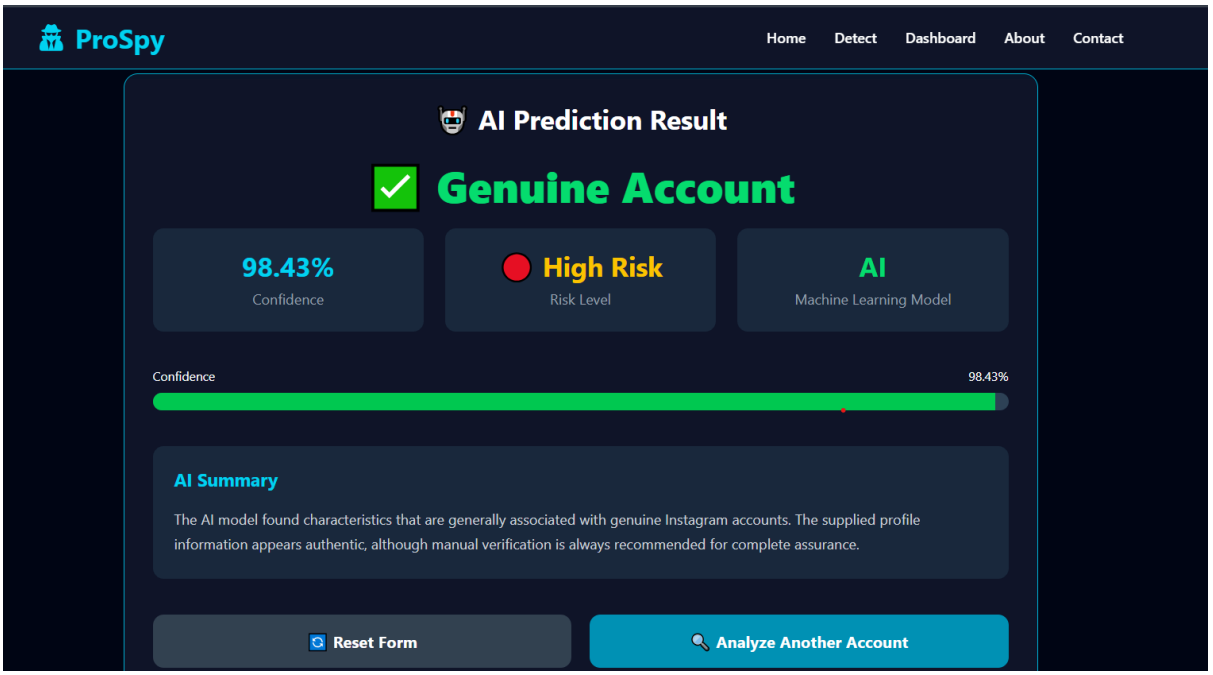
LIVE WEBSITE :

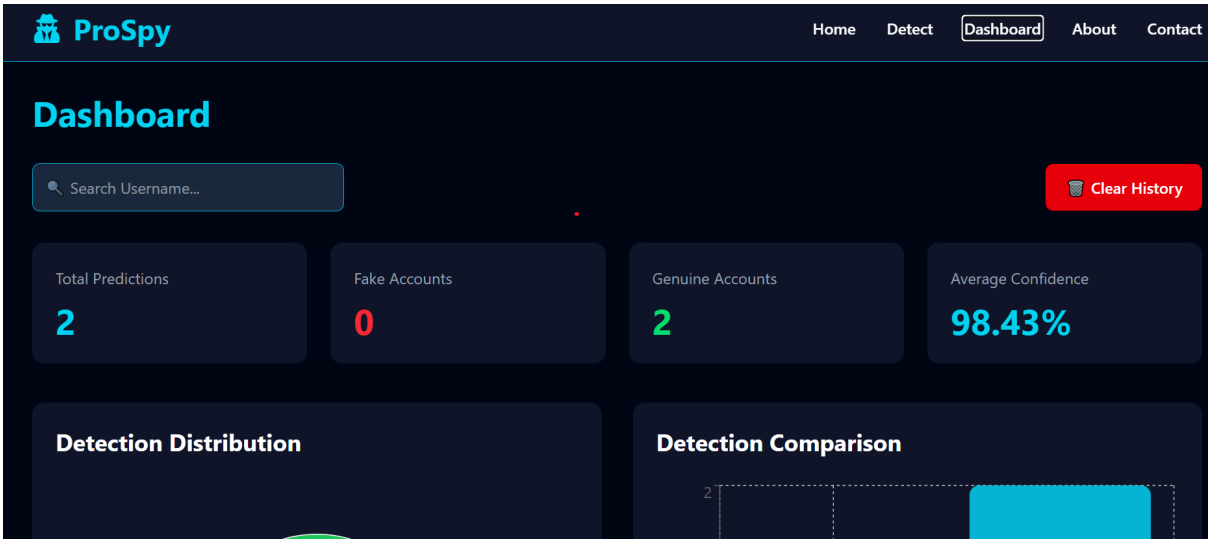
https://pro-spy-fake-account-detector.vercel.app/

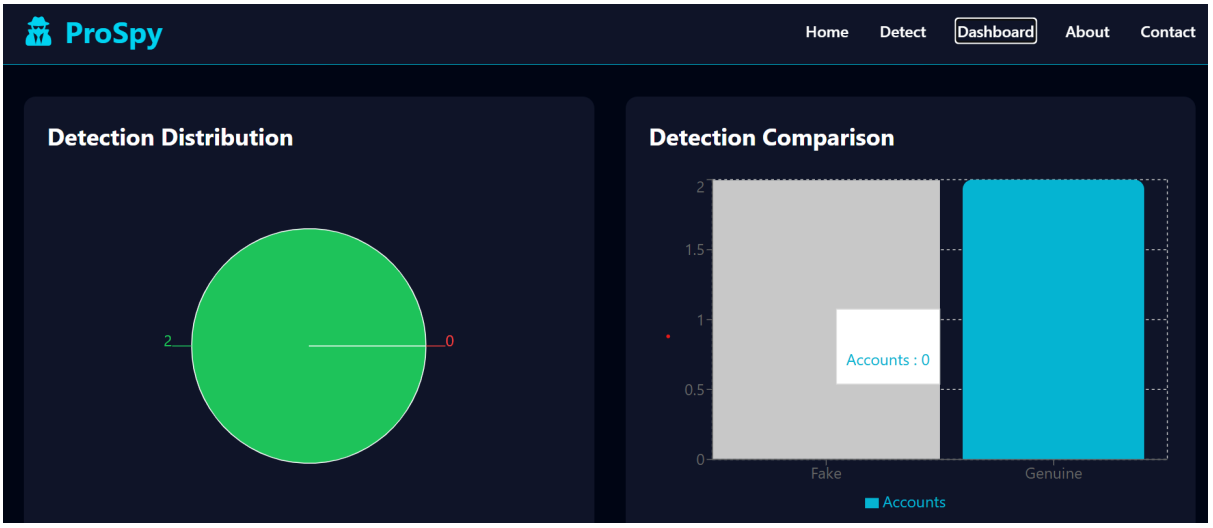



ProSpy
Home
Detect
Dashboard
About
Contact
AI Fake Instagram Account Detection
Fill all account details below and let our AI model determine whether the Instagram profile is Genuine or Fake.
Account Information
Instagram Username
Archisha345
This is only stored in prediction history.
Profile Picture
Yes
Username Randomness
Security & Privacy
Name Equals Username
No
External URL
No
Private Account


ProSpy
Home
Detect
Dashboard
About
Contact
Higher values indicate more random usernames.
Number of Words in Full Name
2
Fullname Randomness
0
Higher values indicate random-looking full names.
Description Length
50
Total Posts
100
Followers
500
Following
300
Analyzing...


ProSpy
Home
Detect
Dashboard
About
Contact
AI Prediction Result
Genuine Account
98.43%
Confidence
High Risk
Risk Level
AI
Machine Learning Model
Confidence
98.43%
AI Summary
The AI model found characteristics that are generally associated with genuine Instagram accounts. The supplied profile information appears authentic, although manual verification is always recommended for complete assurance.
Reset Form
Analyze Another Account


ProSpy
Home
Detect
Dashboard
About
Contact
Dashboard
Search Username...
Clear History
Total Predictions
2
Fake Accounts
0
Genuine Accounts
2
Average Confidence
98.43%
Detection Distribution
Detection Comparison
2


ProSpy
Home
Detect
Dashboard
About
Contact
Detection Distribution
2
0
Detection Comparison
2
1.5
1
0.5
0
Accounts : 0
Fake
Genuine
Accounts


ProSpy
Home
Detect
Dashboard
About
Contact
95%
Model Accuracy
0.9s
Average Prediction Time
AI
Powered Detection
Recent Predictions
Username
Prediction
Confidence
Time
Unknown
Genuine
98.43%
3:48:19 PM
Unknown
Genuine
98.43%
11:37:01 PM